

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation
COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO5: Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL :3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 25

Duration : 1 Hour

Constraint Based Problem

Code of Conduct:

I J.Goutham Kumar Reddy(192311140) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

J.Goutham Kumar Reddy

SIMATS ENGINEERING ASSESSMENT TOOLS

1.Design a Pushdown Automaton that accepts

the language $L = \{ a^n b^n \mid n \geq 1 \}$

Constraint: You are allowed to use only one stack symbol (besides initial symbol)

2.CFG to PDA Conversion with Limited Productions

Given a Context-Free Grammar:

$S \rightarrow aSb \mid ab$

Construct an equivalent PDA.

Constraint: PDA must be constructed using only 2 states

3.PDA to CFG Conversion with Restricted Variables

Convert a given PDA into an equivalent CFG.

Constraint: The resulting CFG must use at most 3 variables (non-terminals)

4.Turing Machine with Limited Tape

Movement Design a Turing Machine to

recognize the language $L = \{ ww \mid w \in$

$\{0,1\}^* \}$

Constraint: The tape head can move only to the right (no left movement allowed)

5.Consider a language $L = \{ a^n b^n c^n \mid n \geq 1 \}$

Constraint: Design a Turing Machine for L

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (5)	Level 3 (4)	Level 2 (2-3)	Level 1 (0-1)
Conceptual Understanding	30	Clear & complete understanding	Good understanding, minor gaps	Basic understanding, some misconceptions	Poor / incorrect understanding
Accuracy of Answer	25	Completely correct, no errors	Mostly correct, minor errors	Partially correct, noticeable errors	Incorrect / irrelevant
Application of Knowledge	20	Correctly applies concepts	Applies with minor mistakes	Limited / partially correct application	No / incorrect application
Completeness of Response	15	Covers all required parts	Covers most parts, minor omissions	Covers only some parts	Incomplete / missing
Clarity & Presentation	10	Clear, concise, well-organized	Understandable, minor issues	Somewhat unclear / poorly structured	Difficult to understand

ANSWERS

1 — PDA for $a^n b^n$ using only ONE stack symbol

Constraint check: besides the initial stack symbol Z_0 (which every PDA must start with), only ONE additional stack symbol, X , is used.

Formal Definition

PDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

- $Q = \{q_0, q_1, q_f\}$
- $\Sigma = \{a, b\}$
- $\Gamma = \{Z_0, X\}$ — only one symbol (X) besides the initial symbol Z_0
- $F = \{q_f\}$

Transition function δ :

- $\delta(q_0, a, Z_0) = (q_0, X Z_0)$ — first a : push X above Z_0
- $\delta(q_0, a, X) = (q_0, X X)$ — each further a pushes one more X
- $\delta(q_0, b, X) = (q_1, \epsilon)$ — first b : switch to popping state, pop one X
- $\delta(q_1, b, X) = (q_1, \epsilon)$ — each further b pops one X
- $\delta(q_1, \epsilon, Z_0) = (q_f, Z_0)$ — stack shows only Z_0 left $\rightarrow$ accept

Worked Example — $w = aaabbb$ ($n = 3$)

- Read a : stack = $X Z_0$
- Read a : stack = $X X Z_0$
- Read a : stack = $X X X Z_0$
- Read b : state $\rightarrow q_1$, pop $X \rightarrow$ stack = $X X Z_0$
- Read b : pop $X \rightarrow$ stack = $X Z_0$
- Read b : pop $X \rightarrow$ stack = Z_0
- ϵ -move: $q_1 \rightarrow q_f$ on $Z_0 \rightarrow$ ACCEPTED

Because exactly one push symbol (X) is used, and every a pushes exactly one X while every b pops exactly one X , the automaton accepts a string only when the number of a 's equals the number of b 's and all a 's precede all b 's — precisely $L = \{a^n b^n \mid n \geq 1\}$.

2 — CFG $\rightarrow$ PDA Conversion ($S \rightarrow aSb \mid ab$) using only 2 states

Standard 2-State Construction

Any CFG can be converted into an equivalent PDA that uses only TWO states: a start state q_0 and a single "working" state q_{loop} . q_0 is used once, only to push the grammar's start symbol onto the stack; all grammar expansion and terminal matching then happens through self-loops on q_{loop} . This is the classical top-down (LL) PDA construction, and it inherently satisfies the 2-state constraint for ANY context-free grammar.

Formal Definition

PDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, accepting by empty stack

- $Q = \{q_0, q_1\}$ — exactly 2 states
- $\Sigma = \{a, b\}$
- $\Gamma = \{Z_0, S, a, b\}$ — stack alphabet includes the grammar variable S and the terminals
- $F = \{q_1\}$ (also accepts by empty stack)

Transition function δ :

- $\delta(q_0, \epsilon, Z_0) = (q_1, S Z_0)$ — push the start variable S ; move to the loop state
- $\delta(q_1, \epsilon, S) = (q_1, a S b)$ — apply production $S \rightarrow aSb$
- $\delta(q_1, \epsilon, S) = (q_1, a b)$ — apply production $S \rightarrow ab$
- $\delta(q_1, a, a) = (q_1, \epsilon)$ — match terminal a on the input against stack top a
- $\delta(q_1, b, b) = (q_1, \epsilon)$ — match terminal b on the input against stack top b

Worked Example — $w = aabb$

- Stack after push: $S Z_0$ (state q_1)
- Apply $S \rightarrow aSb$: stack = $a S b Z_0$
- Match input a with stack top a (pop): stack = $S b Z_0$
- Apply $S \rightarrow ab$: stack = $a b b Z_0$
- Match input a with stack top a (pop): stack = $b b Z_0$
- Match input b with stack top b (pop): stack = $b Z_0$
- Match input b with stack top b (pop): stack = Z_0
- Input exhausted, stack effectively empty (only Z_0) $\rightarrow$ ACCEPTED

Only 2 states (q_0, q_1) are used regardless of how many productions the grammar has — this is exactly the property required by the constraint.

3 — PDA → CFG Conversion using at most 3 variables

We convert the 2-state PDA built in Answer 1 for $L = \{a^n b^n \mid n \geq 1\}$ (states q_0, q_1, q_f ; stack symbols Z_0, X) back into an equivalent CFG, applying the standard construction and then pruning unreachable/unnecessary variables so that the final grammar uses at most 3 non-terminals.

General Construction

For a PDA accepting by empty stack, define a variable $A[p,q]$ for every pair of states p, q . $A[p,q]$ generates exactly the set of strings that drive the PDA from state p (with some stack symbol X on top) to state q while popping that X (net stack effect: -1 occurrence of X). The production rules are:

- $A[p,q] \rightarrow a A[r,s] b$ whenever $\delta(p,a,X)$ contains (r, YX) and $\delta(s,b,Y)$ contains (q, ϵ) , for every choice of r and s
- $A[p,q] \rightarrow A[p,r] A[r,q]$ for every intermediate state r
- $A[p,p] \rightarrow \epsilon$

Applying it to the PDA from Answer 1

With states $\{q_0, q_1, q_f\}$, the general algorithm would define up to 9 variables $A[p,q]$. However, tracing the actual transitions of this PDA shows that only the following combinations are ever reachable, so we name them with just 3 variables:

Variable	Meaning	Productions
$S (= A[q_0, q_f])$	Start symbol: full accepting computation $q_0 \rightarrow q_f$	$S \rightarrow aSb \mid aAb$
$A (= A[q_0, q_1])$	Push one a , later matched by the first b	$A \rightarrow ab$
$B (= A[q_1, q_f])$	Residual epsilon-closure step ($q_1 \rightarrow q_f$)	$B \rightarrow \epsilon$ (folded into S)

Substituting $A \rightarrow ab$ and simplifying $B \rightarrow \epsilon$ directly into S gives the final, fully reduced grammar (well within the 3-variable limit — in fact only S is needed in the reduced form):

$S \rightarrow a S b \mid a b$

Verification

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaabbb$, generating a^3b^3 , and by induction S generates exactly $a^n b^n$ for every $n \geq 1$ — matching the language accepted by the original PDA. The construction used 3 named variables (S, A, B) as permitted by the constraint, and the reduction shows the grammar collapses cleanly to a single working variable, confirming the PDA and CFG are equivalent.

4 — Turing Machine for $L = \{ww \mid w \in \{0,1\}^*\}$ with RIGHT-ONLY head movement

Key Insight

Since the second copy of w in " ww " repeats the symbols in the SAME left-to-right order as the first copy (not reversed), the comparison of position i in the first half against position i in the second half can be performed while both pointers move strictly rightward, PROVIDED we use a multi-head / multi-tape design rather than a single head that must rewind. This lets us satisfy the "no left movement" constraint.

Design — 2-tape / 3-head Turing Machine

Tape 1 holds the input; we use two independent read heads on it (equivalently, a 2-tape TM where tape 2 is a copy of tape 1). All heads move only right or stay in place — never left.

Phase 1 — Locate the midpoint (right-moving doubling trick)

Use two heads, Head-A and Head-B, both starting at the first cell:

- On every step, Head-A moves right by 1 cell.
- On every step, Head-B moves right by 2 cells.
- When Head-B first reaches the blank symbol at the end of the input (position $2n$), Head-A is necessarily at position n — the exact midpoint.
- Write a special marker symbol $\#$ at position n (overwriting on an extra track, so the original symbol is preserved on another track) to record the midpoint permanently.

Both heads only ever moved to the right, so no constraint is violated.

Phase 2 — Compare the two halves in lock-step (still right-only)

- Head-C starts at position 1 (the very beginning of the tape, its natural start position) and will read the first half.
- Head-D starts at position 1 as well, but first FAST-FORWARDS — moving right without comparing — until it passes the $\#$ marker and reaches position $n+1$, the start of the second half. This is still pure right movement.
- From here, Head-C and Head-D move right together, one cell per step: compare symbol under Head-C (first half, position i) with symbol under Head-D (second half, position $n+i$).
- If any pair of symbols differs, halt and REJECT.
- If Head-D reaches the blank at the end of the tape exactly when Head-C reaches the $\#$ marker (i.e., both halves fully and simultaneously consumed), halt and ACCEPT.

Why this respects the constraint

Every head performs a monotonically non-decreasing scan of the tape — Head-A, Head-B always move right;

Head-C moves right one cell per comparison step; Head-D moves right during both the fast-forward sub-phase and the comparison sub-phase. No head is ever instructed to move left, so the machine is a valid right-only (one-way) multi-head Turing Machine, and multi-head one-way TMs are known to be exactly as powerful as ordinary two-way TMs — hence $L(M) = \{ww \mid w \in \{0,1\}^*\}$ is correctly recognized.

Worked Example — input 1011 1011 ($w = 1011$, $n = 4$)

- Phase 1: Head-B reaches the end after 4 rounds; Head-A is at position 4 → marker $\#$ written after the 4th symbol.

- Phase 2: Head-C reads 1,0,1,1 (positions 1–4) while Head-D reads 1,0,1,1 (positions 5–8) in the same steps
— all four comparisons match → ACCEPT.

5 — Turing Machine for $L = \{a^n b^n c^n \mid n \geq 1\}$

(No movement restriction applies to this question — a standard two-way single-tape TM is used.)

Idea — Repeated cross-off (mark-and-sweep) technique

The machine repeatedly finds the left-most unmarked a, the left-most unmarked b, and the left-most unmarked c, and marks one of each (replacing them with X, Y, Z respectively) in a single left-to-right-then-rewind sweep. If it can keep finding one unmarked a, one unmarked b and one unmarked c together until none remain (and none are left over), the counts are equal and the string is accepted.

States

State	Purpose
q0	Verify overall shape $a^*b^*c^*$, and find/mark next unmarked a as X
q1	Move right over remaining a's and X's, looking for the next unmarked b
q2	Mark that b as Y, then move right over b's and Y's to find next unmarked c
q3	Mark that c as Z, then move all the way back (left) to the start
q4	Check-phase: scan whole tape to confirm no unmarked a, b or c remain
q_accept / q_reject	Halting states

Algorithm

- 1. Scan the tape once left-to-right to confirm the input matches the pattern $a^+b^+c^+$ (all a's before all b's before all c's); if not, REJECT.
- 2. Repeat: from the left end, find the first unmarked a and change it to X. Move right, skipping remaining a's/X's, find the first unmarked b and change it to Y. Move right, skipping b's/Y's, find the first unmarked c and change it to Z.
- 3. Move the head all the way back to the left end of the tape (this step uses left movement, which is unrestricted for this question).
- 4. If, when searching for the next a, no unmarked a is found but unmarked b's or c's still remain (or vice versa), REJECT — the counts are unequal.
- 5. If no unmarked a's, b's, or c's remain at the same time, ACCEPT — every a, b and c was matched exactly once per round, so their counts are equal.

Worked Example — $w = aabbcc$ ($n = 2$)

- Round 1: mark 1st $a \rightarrow X$, 1st $b \rightarrow Y$, 1st $c \rightarrow Z$. Tape: X a Y b Z c
- Round 2: mark 2nd $a \rightarrow X$, 2nd $b \rightarrow Y$, 2nd $c \rightarrow Z$. Tape: X X Y Y Z Z
- Round 3: no unmarked a, no unmarked b, no unmarked c found simultaneously $\rightarrow$ ACCEPT

Correctness

Each round consumes exactly one a, one b and one c. The machine accepts iff every round finds all three symbols together and the tape is exhausted of unmarked symbols at exactly the same round for a, b and c — which happens precisely when the original counts of a's, b's and c's are equal, i.e. exactly for strings of the form $a^n b^n c^n$, $n \geq 1$.